Testing, Quality Assurance & Readiness Plan

Document 07 of 07

Purpose
Close the handover package with an evidence-based view of existing automated checks, what those checks do and do not prove, and a prioritized quality plan for future changes and releases.

Repository snapshot	rd2080/Bulk_App_Personalised, default branch main; test files and CI workflow reviewed on 21 October 2026.
Current verification	Test source inspected; no test suite or CI run was executed for this document. No pass/fail claim is made.
Title assumption	The package outline naming Document 07 was not available in this task context. QA/readiness is the working final-document title.
Status terms	**Existing coverage** means a corresponding test module exists. **Proposed** means a recommended check not confirmed in current automated tests.

Readiness summary

GitHub Actions defines a CI job triggered for pull requests and pushes to main. It installs requirements and pytest under Python 3.11, then runs pytest -q. The repository includes tests for inventory, profile, check-in, recipe library/CSV, meal planning, deterministic commands, request building and an older SQLite service path. Test modules frequently mock database connections; their presence does not establish end-to-end Neon behavior or current CI health.

Implementation status in this document is based on source and test-file inspection. It is not a test report.

1. Existing automated coverage

Area / test module	Evidence visible in tests	Coverage boundary
Food inventory	tests/test_food_inventory.py checks add, duplicate rejection, availability and update repository behavior.	Repository calls are mocked; no real Neon or Streamlit page workflow proven.
User profile	tests/test_user_profile.py checks exact fields, normalization, invalid values and repository reads/writes.	Does not prove live DB migration state or browser rendering.
Morning check-in	tests/test_morning_checkin.py checks date upsert, weight history write and invalid values rejected before DB calls.	Mocked connection; transaction behavior and live constraints need integration verification.
Recipe library and CSV	tests/test_recipe_library.py covers controlled values, numeric ranges, nutrition scaling, CSV parsing/import templates and rollback-oriented behavior.	Uses fake connection for many cases; no real Neon import round trip established.
Meal plan	tests/test_meal_plan.py checks creation, ordering, selected-plan scoping, invalid values and deletion/listing.	Mocked repository DB; no Streamlit UI, concurrent writes or live constraints proven.
Deterministic commands	tests/test_meal_commands.py checks labeled meals, add/replace/remove parsing, ambiguity/clarification and application outcomes.	Rule coverage is finite; browser speech recognition and full UI flow not automated by this module.
Request builder	tests/test_request_builder.py checks fresh profile reads, context envelope, empty task and missing profile errors.	No provider call or generated meal suggestion exists to verify.
Services (legacy path)	tests/test_services.py uses SQLite temporary DB to test older daily-plan/service persistence and provider selection.	Potential mismatch with current Neon repositories and documented feature model; confirm intended maintenance or deprecation.

Coverage status is inferred from the reviewed test source. Tests were not run, and no GitHub Actions run result was retrieved.

2. Test strategy for changes

Use the lowest-cost test that can establish the required behavior, then add integration or UI checks where boundaries cross. Keep test data isolated and never point routine automated writes at production.

Level	Purpose	Target examples
Unit	Fast deterministic validation and business rules.	Field bounds; duplicate/reference checks; recipe nutrition scaling; command parser; profile request shape.
Repository / DB integration	Verify SQL, constraints, transaction and foreign-key behavior against an isolated PostgreSQL/Neon branch.	Check-in + weight atomic upsert; recipe CSV rollback; plan row scoping; migration constraints; concurrency on unique dates.
UI workflow	Verify page interaction, feedback and session behavior.	Manual add/edit/delete; date selection; CSV preview/confirm; voice text fallback; save errors retain draft.
Browser compatibility	Verify browser-dependent microphone behavior and graceful fallback.	Supported browsers, denied permission, no speech result, HTTPS, manual text submission.
Release smoke	Check candidate app against intended environment after migration/release.	Read profile, inspect date plan, perform approved test-branch write and reload, check logs for errors.

3. Recommended coverage priorities

Priority	Recommended check	Why it matters
P0	Run current CI on every PR/main commit; report exact commit SHA and workflow result.	Confirms the defined automated suite executes on candidate code.
P0	Validate test_services.py ownership: migrate it to current Neon architecture if still supported, or clearly mark/remove legacy service expectations.	Avoid false confidence from tests of a separate SQLite model/provider flow.
P0	Add isolated PostgreSQL integration coverage for migrations 001-008 and repository transaction boundaries.	Current mocked tests do not establish schema compatibility or actual rollback.
P1	Add end-to-end tests for profile/check-in/plan/recipe page workflows and error states.	Exercise form validation, reruns, date scoping, confirmation and visible success/failure messaging.
P1	Add edge cases for command parser: empty input, generic snack, repeated labels, duplicate existing slots, unsupported punctuation, and no mutation on ambiguity.	Parsing mistakes can alter a saved day plan.
P1	Test CSV import atomicity with real transaction failure after at least one attempted write and confirm no partial state.	Import replaces ingredient definitions and must remain all-or-nothing.
P2	Add browser matrix and accessibility checks for keyboard use, focus, labels, error announcements and speech fallback.	UI and speech input depend on device/browser capabilities.
P2	Add nonfunctional checks for dependency/security updates, secret scanning and database connection behavior.	CI workflow currently establishes unit test execution only.

4. High-value acceptance scenarios

QA-01 CI Given a pull request or push to main, the configured workflow installs the pinned-range dependencies and runs pytest -q; the result is visible for the tested commit SHA.

QA-02 Profile Given missing, extra or out-of-range profile data, validation rejects the save and persists no partial update.

QA-03 Check-in atomicity Given a database error during paired check-in and weight writes, the transaction rolls back both changes.

QA-04 CSV atomicity Given an invalid reference or simulated database exception, no recipe or ingredient rows are partially imported.

QA-05 Plan isolation Given two dates with plans, editing or deleting a meal on one date cannot change the other date.

QA-06 Ambiguous command Given generic “snack” or multiple candidate meals, applying text returns clarification and leaves plan rows unchanged.

QA-07 Browser fallback Given unsupported speech recognition or denied microphone permission, manual text entry remains usable and empty input is not applied.

QA-08 Migration verification Given a clean isolated PostgreSQL database, documented migrations apply in order and expected constraints/relationships are queryable.

QA-09 Release smoke Given candidate app code and matching migrations on staging, critical reads and an approved write/reload smoke check complete before release acceptance.

5. Test data and environment rules

- Use synthetic profile, food, recipe and meal content in automated tests. Avoid personal health, diet or microphone transcripts in logs, fixtures or screenshots unless explicitly approved and safely de-identified.
- Use temporary SQLite only for tests whose target is specifically the SQLite service module; use isolated PostgreSQL for Neon-backed repository and migration integration tests. Do not substitute one for the other.
- Automated DB tests must have an explicit test DATABASE_URL/branch guard and fail closed if configuration points to a production environment. Clean up only records created by the test.
- Use deterministic dates and timezone assumptions in tests. Freeze or inject clocks for date-sensitive UI and reminder features if introduced.

- Mock browser speech and external AI providers at unit level; separately test their integration boundaries and manual fallback without sending private content.

6. Exit criteria for a release

Gate	Required evidence
Code	Review completed; candidate SHA recorded. CI run for candidate SHA passes, or approved exception documented.
Data	Migration order and staging schema verified; backup/recovery prerequisite confirmed.
Workflows	Critical smoke scenarios pass in intended staging/host environment.
Failure paths	Database configuration, validation, CSV, and voice fallback failures show no false success.
Operations	Release record includes owner, environment, applied migrations, smoke outcomes and rollback contact.

7. Known gaps and handover follow-ups

- No test run or CI status was checked for this document. Existing workflow configuration is not evidence of a passing current branch.
- Unit tests around repository functions often mock the database connection. Add real PostgreSQL integration coverage before treating constraints, migrations and transaction rollback as release-proven.
- The legacy SQLite tests exercise another application/service model; confirm whether those paths remain supported or are historical code.
- No automated Streamlit page/browser end-to-end suite was identified in the repository evidence reviewed.
- Browser Web Speech quality and privacy are vendor/browser dependent. Keep manual text path in all voice-related releases and state supported browsers from actual tests.
- Future actual meal logging, reminders, charts, nutrition analysis and AI suggestions require new test fixtures and quality gates when implemented; they are not covered by current tests.

8. Evidence register

Current GitHub main evidence: .github/workflows/ci.yml; tests/test_food_inventory.py; tests/test_meal_commands.py; tests/test_meal_plan.py; tests/test_morning_checkin.py; tests/test_recipe_library.py; tests/test_request_builder.py; tests/test_services.py; tests/test_user_profile.py; README.md; docs/voice-input.md. The local project mirror is not fully synchronized with GitHub main.

The test suite was not run. The GitHub Actions workflow history was not inspected. No application, database, browser, coverage report, security scanner or accessibility audit was executed. Recommendations and acceptance scenarios in this PDF are proposed future checks, not current results. The PDF was text-checked and rendered for visual inspection.

End of Document 07